

FT-Transformer: Resilient and Reliable Transformer with End-to-End Fault Tolerant Attention

Huangliang Dai
University of California, Riverside
Riverside, USA
huangliang.dai@email.ucr.edu

Shixun Wu
University of California, Riverside
Riverside, USA
swu264@ucr.edu

Jiajun Huang
University of South Florida
Tampa, USA
jiajunhuang@usf.edu

Zizhe Jian
University of California, Riverside
Riverside, USA
zjian106@ucr.edu

Yue Zhu
University of California, Riverside
Riverside, USA
yzhu303@ucr.edu

Haiyang Hu
University of California, Riverside
Riverside, USA
hhu064@ucr.edu

Zizhong Chen
University of California, Riverside
Riverside, USA
chen@cs.ucr.edu

Abstract

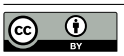
Transformer models rely on High-Performance Computing (HPC) resources for inference, where soft errors are inevitable in large-scale systems, making the reliability of the model particularly critical. Existing fault tolerance frameworks for Transformers are designed at the operation level without architectural optimization, leading to significant computational and memory overhead, which in turn reduces protection efficiency and limits scalability to larger models. In this paper, we implement module-level protection for Transformers by treating the operations within the attention module as a single kernel and applying end-to-end fault tolerance. This method provides unified protection across multi-step computations, while achieving comprehensive coverage of potential errors in the nonlinear computations. For linear modules, we design a strided algorithm-based fault tolerance (ABFT) that avoids inter-thread communication. Experimental results show that our end-to-end fault tolerance achieves up to 7.56× speedup over traditional methods with an average fault tolerance overhead of 13.9%.

CCS Concepts

• **Computing methodologies** → **Parallel algorithms**; **Machine learning**; • **Computer systems organization** → **Dependable and fault-tolerant systems and networks**.

Keywords

Fault Tolerance, Transformer, Attention Mechanism, Reliability, ABFT, Deep Learning



This work is licensed under a Creative Commons Attribution 4.0 International License.

SC '25, St Louis, MO, USA

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1466-5/25/11
https://doi.org/10.1145/3712285.3759853

ACM Reference Format:
Huangliang Dai, Shixun Wu, Jiajun Huang, Zizhe Jian, Yue Zhu, Haiyang Hu, and Zizhong Chen. 2025. FT-Transformer: Resilient and Reliable Transformer with End-to-End Fault Tolerant Attention. In *The International Conference for High Performance Computing, Networking, Storage and Analysis (SC '25), November 16–21, 2025, St Louis, MO, USA.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3712285.3759853

1 Introduction

Transformer models [40] have revolutionized the field of deep learning, demonstrating remarkable capabilities in natural language processing [18, 49], computer vision [20], and multimodal learning [45]. In recent years, the rapid advancement of large language models (LLMs) has spurred extensive research on Transformers, leading to diverse specialized models, such as GPT [34], BERT [9] and T5 [29]. These models utilize the attention mechanism to capture long-range dependencies and complex relationships, achieving outstanding performance across a wide range of applications.

With the ongoing advancements in model capabilities, Transformers are increasingly integrated in autonomous agents [3, 26] and complex reasoning [42, 48] scenarios that involve both human interaction and real-world engagement, placing more stringent demands on inference reliability. However, Transformer inference is frequently affected by hardware soft errors, making it challenging to ensure computational reliability. Previous studies have shown that soft errors occur far more frequently than other faults, with rates reaching up to 50,000 failures-in-time (FIT)/chip if unprotected [1]; moreover, the error rate may escalate due to hardware aging and thermal cycling [2, 31]. Google's technical report [39] indicates that the silent data corruption caused by soft errors is likely to affect Gemini computation every week or two.

Soft errors can severely affect Transformer inference, potentially leading to system crashes or silent corruption of computation results. A series of research works have investigated the impact of soft errors in accelerator hardware on deep learning model inference performance, revealing that even a single soft error can degrade prediction accuracy, with its severity influenced by data

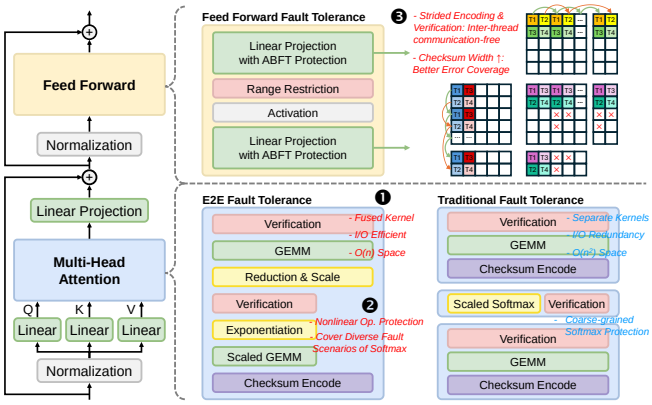


Figure 1: FT-Transformer framework, where end-to-end fault tolerant attention protects both linear and nonlinear computations within a single kernel, and the feed forward module is safeguarded using strided ABFT.

type and model architecture [22, 35]. More recent studies [16, 46] have further explored the error resilience of Transformer models. Specifically, [46] finds that injecting bit-flip errors into Transformers leads to an accuracy drop of approximately 9%–20% at a bit error rate of 1e-9. The analysis in [16] evaluated BERT's reliability in classification and question-resolution tasks, showing that an individual bit-flip can cause up to a 70% loss in accuracy, and certain bit positions consistently lead to unreliable model inferences.

To maintain the reliability of Transformer computations, various fault tolerance frameworks have been introduced. In [46], Xue et al. proposed a framework that integrates block-wise ABFT and range restriction to protect general matrix multiplication (GEMM) and softmax operations in vision Transformers. In [27], Liang et al. designed an attention checker to detect and correct INF, NaN, and near-INF values in the GEMM operations of the attention mechanism. However, these frameworks are designed to protect individual operations, and directly applying them to Transformers incurs significant overhead, particularly for the attention mechanism. These frameworks implement fault tolerance on decoupled GEMM and softmax kernels, invoking three separate kernels to complete a single attention computation. Due to the quadratic time and memory complexity of the attention mechanism, storing intermediate computation results of GEMM and softmax would result in substantial memory consumption. Additionally, redundant memory access and excessive kernel launch overhead make these fault tolerance frameworks highly inefficient.

Driven by this motivation, we propose the FT-Transformer framework in Figure 1. This framework adopts an end-to-end fault tolerant attention (EFTA) design that treats attention computation as a unified operation, implementing error detection and correction for GEMMs and softmax within a single kernel. The EFTA prevents memory access to $O(n^2)$ intermediate results by tiling the attention scores and performing in-place computations. This enables efficient fault tolerance for long-sequence inference. EFTA also simplifies the fault tolerance process within the fused kernel. By reordering the execution sequence of computation and fault tolerance, the three verification stages are reduced to two. Meanwhile, to enhance the protection of softmax computation, we design selective neuron value restriction (SNVR). This method applies varying fault tolerance constraints based on computational importance, employing extended checksums to protect exponential computations and range restriction to safeguard reduction operations. Compared to traditional DMR approaches, it significantly reduces protection overhead, while offering finer fault tolerance granularity than range restriction. For linear feed-forward modules, we design tensor checksums aligned with the thread-level data layout of tensor cores. By performing checksum encoding and verification via strided accumulation, we realize an strided ABFT scheme that requires no inter-thread communication.

In this paper, the same protection mechanism is applied to both the linear modules and the linear computations within EFTA. Therefore, the following sections primarily focus on the design of EFTA. The main contributions of this paper can be summarized as follows.

- **The first end-to-end fault tolerance framework for attention mechanism**: We design an end-to-end fault tolerant attention to protect computations against soft errors, eliminating the $O(n^2)$ memory overhead from intermediate data access and reordering computations for unified error verification across multiple steps. Our design enables reliable long-sequence inference with minimal overhead under erroneous conditions.
- **Hybrid scheme covering all error scenarios**: The end-to-end framework restructures the computation flow and introduces rescale and reduce max operations. We propose a hybrid fault tolerance scheme that integrates strided ABFT and selective neuron value restriction (SNVR) for this paradigm to ensure comprehensive error coverage. This scheme employs checksum reuse, enabling ABFT to protect GEMM, exponentiation, and rescale operations, while applying SNVR to enhance fault tolerance in reduce-max and row-sum operations by constraining neuron values based on computational significance.
- **Strided ABFT optimized for Tensor Cores**: To optimize fault tolerance efficiency on Tensor Cores, we develop an architecture-aware ABFT, using tensor checksum for computation verification. The strided ABFT leverages the thread-data mapping of matrix multiply-accumulate (MMA) instructions to perform strided computations, enabling intra-thread checksum encoding and error correction. With additional checksum values, this design demonstrates better fault tolerance in the presence of multiple errors.
- **Efficient Fault Tolerance**: We evaluate EFTA under the attention settings of medium and large models. Experimental results show that EFTA incurs an average fault tolerance overhead of 13.9%, while achieving a 3.69× to 7.56× speedup compared to existing methods. Simulations on Transformer models indicate that EFTA is efficient in error detection and correction, incurring overheads of 4.7% and 9.1% respectively.

2 Background and Related Works

In this section, we provide an overview of the efficient attention mechanisms adopted in state-of-the-art Transformer models. We also define the fault model used in this study and review existing techniques in fault tolerant neural network design. The notations and symbols used in this paper are summarized in Table 1.

Table 1: Notations and Symbols

Notation	Description
$\mathbf{Q}$	Query, in $N \times d$
$\mathbf{K}$	Key, in $N \times d$
$\mathbf{V}$	Value, in $N \times d$
$\mathbf{S}$	Attentions Score, in $N \times N$
ℓ	Rowsum Vector, in N
$\mathbf{P}$	Attentions Probability, in $N \times N$
$\mathbf{O}$	Output, in $N \times d$
N	Sequence Length
d	Head Dimension
n	iteration index

2.1 Efficient Attention Mechanism Implementation

Attention is the core of Transformer models, enabling efficient handling of non-local dependencies and parallel sequence processing. However, this mechanism has a time and space complexity of $O(n^2)$, incurring significant computational and memory overhead during inference with long sequences. To address this bottleneck and implement fast and memory-efficient inference in transformer models, several attention algorithms have been proposed, including sparse attention [24, 37], linear attention [23, 41], and flash attention [7, 8]. Among these methods, only flash attention implements standard exact attention, whereas the others achieve attention through approximation computations. This characteristic allows flash attention to be seamlessly integrated into most transformer models without altering the layer structure. The flash attention algorithm is formalized in Equations (1)-(7).

In flash attention, the Score tensor $\mathbf{S}$ is partitioned along the row dimension into segments $\left[\mathbf{S}^{(1)}\ \mathbf{S}^{(2)}\ \cdots\ \mathbf{S}^{(n)}\right]$, where each $\mathbf{S}^{(i)} \in \mathbb{R}^{N \times B_c}$ for $i = 1, 2, ..., n$, and $B_c = N/n$. The Value tensor $\mathbf{V}$ is correspondingly divided along the column dimension into into n segments, where each segment $\mathbf{V}^{(i)} \in \mathbb{R}^{B_r \times d}$ for $i = 1, 2, \ldots, n$, and $B_r = B_c$. Through this tiling approach, the global softmax can be decomposed into several local softmax operations, enabling the iterative update of the Attention tensor $\mathbf{O}$. The initial update $\mathbf{O}^{(1)}$ can be calculated as follows:

$$\ell^{(1)} = \text{rowsum}(e^{\mathbf{S}^{(1)}}) \in \mathbb{R}^N \tag{1}$$

$$\widetilde{\mathbf{P}}^{(1)} = \text{diag}(\ell^{(1)})^{-1} e^{\mathbf{S}^{(1)}} \in \mathbb{R}^{N \times B_c} \tag{2}$$

$$\mathbf{O}^{(1)} = \widetilde{\mathbf{P}}^{(1)}\mathbf{V}^{(1)} = \text{diag}(\ell^{(1)})^{-1} e^{\mathbf{S}^{(1)}}\mathbf{V}^{(1)} \in \mathbb{R}^{N \times d} \tag{3}$$

In the second iteration, the local rowsum of $\mathbf{S}^{(2)}$ is accumulated on the previous normalization constant $\ell^{(1)}$, yielding the total rowsum for the first two segments. Scaling $\mathbf{O}^{(1)}$ and adding the local attention tensor from segment two gives $\mathbf{O}^{(2)}$.

$$\ell^{(2)} = \ell^{(1)} + \text{rowsum}(e^{\mathbf{S}^{(2)}}) \in \mathbb{R}^N \tag{4}$$

$$\widetilde{\mathbf{P}}^{(2)} = \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(2)}} \in \mathbb{R}^{N \times B_c} \tag{5}$$

$$\mathbf{O}^{(2)} = \text{diag}(\ell^{(1)}/\ell^{(2)})^{-1}\mathbf{O}^{(1)} + \widetilde{\mathbf{P}}^{(2)}\mathbf{V}^{(2)} \in \mathbb{R}^{N \times d} \tag{6}$$

By iteratively applying the update process, the global rowsum is aggregated across all segments, which leads to the global normalization constant $\ell^{(n)}$. The final attention tensor is then computed using this normalization factor.

$$\begin{aligned}\mathbf{O} &= \text{diag}(\ell^{(n-1)}/\ell^{(n)})^{-1}\mathbf{O}^{(n-1)} + \widetilde{\mathbf{P}}^{(n)}\mathbf{V}^{(n)} \\ &= \text{diag}(\ell^{(n)})^{-1}\sum_{i=1}^{n-1}(e^{\mathbf{S}^{(i)}}\mathbf{V}^{(i)}) + \text{diag}(\ell^{(n)})^{-1}e^{\mathbf{S}^{(n)}}\mathbf{V}^{(n)} \\ &= \text{diag}(\text{rowsum}(e^{\mathbf{S}}))^{-1}e^{\mathbf{S}}\mathbf{V} \in \mathbb{R}^{N \times d}\end{aligned} \tag{7}$$

2.2 Fault Model

FT-Transformer is designed to provide resilient and reliable inference against soft errors, which silently corrupt data by bit-flips and lead to incorrect inference results without any visible failure. Soft errors may arise from various hardware components, depending on the location of the fault: 1) memory faults, which affect large storage components, including main memory and cache; 2) computing unit faults, which occur in arithmetic or control logic components such as ALUs and FPUs, leading to incorrect computation outcomes; 3) interconnect faults, which arise in buses, interconnects, or NoC components and may cause data corruption or loss during transfers between compute units and memory.

This study focuses on detecting and correcting soft errors from computing unit faults, assuming that memory faults are mitigated by Error Correction Code (ECC) [14] and interconnect faults are managed by FT-MPI [13]. Furthermore, FT-Transformer is developed based on a single event upset (SEU) assumption [32], where at most one computational error is expected during each detection and correction cycle. Due to the low occurrence rate of multiple soft errors enabled by short fault detection intervals, this assumption is considered valid and has been adopted in previous studies [36, 43].

2.3 Fault Tolerance in Deep Learning

Various fault tolerance methods have been explored in deep learning to improve model robustness and reduce the impact of computational errors on prediction accuracy. These methods can be categorized as follows: 1) Modifying the model architecture, including approaches such as adding additional fault tolerant modules [10, 11, 28] or duplicating critical layers [5, 6, 12, 33]; 2) Retraining the model parameters, introducing artificial errors into the dataset or layer outputs during training. [4, 17, 19]; 3) Restricting neuron values [15, 47]. In addition to these explorations in model intrinsic resilience, many methods from the high-performance computing (HPC) domain can also be applied to the fault tolerance design of learning networks. Classical approaches include dual modular redundancy (DMR) [38], error-correcting code (ECC) [14], and algorithm-based fault tolerance (ABFT) [21, 43, 44]. Among these three approaches, ABFT is commonly adopted in deep learning models [30, 50] due to its low overhead and compatibility with different model architectures. The fundamental principle of ABFT is to utilize redundant information encoded in checksums to verify the correctness of matrix multiplications. The encoding process can be mathematically formulated as:

$$\mathbf{A} \xrightarrow{\text{encode}} \mathbf{A}^c := \begin{bmatrix}\mathbf{A} \\ \mathbf{c}_1\mathbf{A} \\ \mathbf{c}_2\mathbf{A}\end{bmatrix} \in \mathbb{R}^{(M+2)\times K} \tag{8}$$

$$\mathbf{B} \xrightarrow{\text{encode}} \mathbf{B}^r := \begin{bmatrix}\mathbf{B} & \mathbf{B}\mathbf{r}_1 & \mathbf{B}\mathbf{r}_2\end{bmatrix} \in \mathbb{R}^{K\times(N+2)} \tag{9}$$

where $\mathbf{c}_1 = [1, 1, 1, ..., 1]^\top$ and $\mathbf{c}_2 = [1, 2, 3, ..., M]^\top$ denote the column checksum weights, while $\mathbf{r}_1 = [1, 1, 1, ..., 1]$ and $\mathbf{r}_2 = [1, 2, 3, ..., N]$ represent the row checksum weights. The encoded matrices $\mathbf{A}^c$ and $\mathbf{B}^r$ are then multiplied to produce the verification matrix $\mathbf{C}^f$, which equals to original multiplication result $\mathbf{C}$ augmented with two checksum columns $\mathbf{C}^{c1}$, $\mathbf{C}^{c2}$, and two checksum rows $\mathbf{C}^{r1}$, $\mathbf{C}^{r2}$. To detect errors in the matrix $\mathbf{C}$, its column values are weighted and summed to produce $\mathbf{C}^{c1'} = \sum_{i=0}^{M-1} \mathbf{C}_{i,0:N-1}$ and $\mathbf{C}^{c2'} = \sum_{i=0}^{M-1} (i+1) \times \mathbf{C}_{i,0:N-1}$. Then $\mathbf{C}^{c1'}$ is compared with $\mathbf{C}^{c1}$. If they are not equal to each other at position j, the error can be located by row index $i = \frac{\mathbf{C}^{c2'}[j] - \mathbf{C}^{c2}[j]}{\mathbf{C}^{c1'}[j] - \mathbf{C}^{c1}[j]}$ and column index j, and it can be corrected by adding $\mathbf{C}^{c1'}[j] - \mathbf{C}^{c1}[j]$ to element $\mathbf{C}[i][j]$. The row checksum correction follows a similar procedure.

3 End-to-End Fault Tolerant Attention

In this section, we present the first end-to-end fault tolerant attention (EFTA) mechanism for transformer models, achieving efficient error detection and correction within a single kernel, while mitigating potential memory faults and supporting reliable inference on long input sequences.

3.1 Operation-level protection for Attention with decoupled kernels

Existing fault-tolerance frameworks for attention primarily apply a single form of protection, either linear or nonlinear computations, resulting in limited error coverage. In this subsection, we first present a preliminary analysis toward comprehensive fault tolerance in attention with these operation-level protection mechanisms. The ABFT for matrix-matrix multiplication can be utilized to protect the similarity computation of tensors $\mathbf{Q}$ and $\mathbf{K}$, as well as the weighted aggregation of tensors $\mathbf{P}$ and $\mathbf{V}$, owing to their reliance on GEMM. For nonlinear operations, DMR can be utilized to verify the integrity of tensor $\mathbf{P}$ following the row softmax operation. The exponential operation and normalized weight calculation in the softmax function can be validated using the following two equations, where n denotes the iteration index of the repeated computation.

$$\mathrm{DMR}(e^{\mathbf{S}}) = \frac{e^{\mathbf{S}}_n + e^{\mathbf{S}}_{n-1}}{2}, \quad |e^{\mathbf{S}}_n - e^{\mathbf{S}}_{n-1}| < \epsilon \tag{10}$$

$$\mathrm{DMR}(\mathbf{P}) = \mathbf{P}_n, \quad |\mathrm{rowsum}(\mathbf{P}_n) - \mathbf{c}_1| < \epsilon \tag{11}$$

The DMR validation kernel iteratively executes these two operations until the difference between consecutive computations satisfies a pre-defined error tolerance.

• **Operation-level Protection Framework.** The block partitioning of the operation-level protection framework for the attention mechanism is illustrated in Figure 2. The attention computation is divided into three kernels executed sequentially, with each kernel accessing the High Bandwidth Memory (HBM) for data reads and writes. These kernels break down computation and protection into tiled blocks, enabling finer-grained fault tolerance and in-kernel parallelism. The green blocks represent operations executed by a single cooperative thread array (CTA) on the GPU, whereas the blue blocks indicate parallel workloads distributed across other CTAs.

All tensors involved in the attention computation have the shape $batch \times num_head \times seq_len \times feature_dim$. Since the $batch$ and

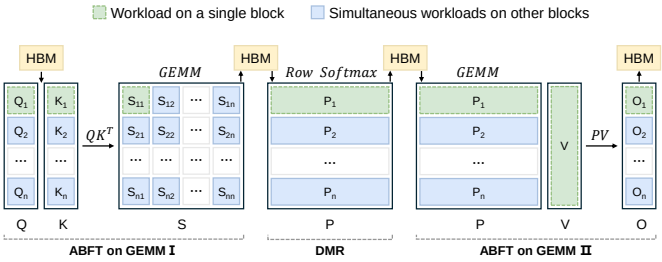


Figure 2: Operation-level fault tolerant attention at the block granularity.

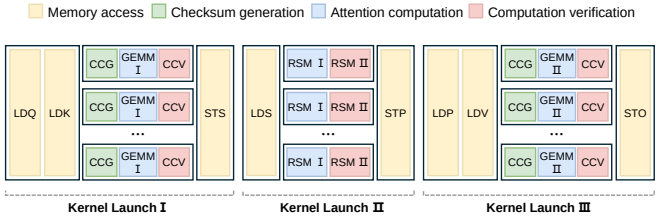


Figure 3: Workflow of the operation-level fault tolerance with decoupled kernels.

num_head dimensions are inherently parallel, block tiling is applied exclusively to the seq_len and $feature_dim$ dimensions. For the ABFT on GEMM I, $\mathbf{S}$ is divided into submatrices $\mathbf{S}_{ij}$ of size $B_r \times B_c$. Similarly, $\mathbf{Q}$, $\mathbf{K}^\top$ are tiled along the rows and columns, respectively, using the corresponding block sizes. Each time $\left\lceil \frac{seq_len}{B_r} \right\rceil \times \left\lceil \frac{seq_len}{B_c} \right\rceil$ blocks are processed in parallel, computing $\mathbf{Q}_i \mathbf{K}_j^\top$ with ABFT protection. For the DMR applied to row softmax, tiling is performed along the rows, with each block computing $\mathbf{P}_i$ using equations (10), (11). The ABFT on GEMM II computes $\mathbf{P}_i \mathbf{V}$ within each block.

• **Operation-level Fault Tolerance Workflow.** Figure 3 depicts the workflow designed for fault tolerant computations within the operation-level protection framework. The yellow sections represent memory operations, with LD and ST indicating tensor data loading and storing. The green sections denote checksum encoding computations, where CCG stands for computation checksum generation. The blue sections represent original attention operations, where RSM denotes row softmax. The red sections indicate the overhead from verification, with CCV representing computation checksum verification.

The fault tolerance workflow proceeds as follows. First, the ABFT-GEMM kernel for $\mathbf{QK}^\top$ is launched. Within the kernel, data blocks of $\mathbf{Q}_i$ and $\mathbf{K}_j$ are loaded from HBM to the on-chip memory of each streaming multiprocessor (SM) following the tiling scheme outlined in the operation-level protection framework. For simplicity, details of data parallelism across the $batch$ and num_head dimensions, as well as memory access patterns in shared memory and registers, are omitted. Subsequently, parallel checksum generation is executed in the tensor cores, producing row and column checksums for each block, denoted as $\mathbf{c}_1\mathbf{Q}_i$, $\mathbf{c}_2\mathbf{Q}_i$ and $\mathbf{K}_j\mathbf{r}_1$, $\mathbf{K}_j\mathbf{r}_2$. These checksums, along with the original data are then processed by the block-level GEMM operations to compute the corresponding $\mathbf{S}_{ij}$. After the computation is completed, intra-block data validation is conducted.

If errors are detected, the checksums are utilized to correct computation errors. Once the correction is finalized, the result tensor **S** is written back to the HBM, and the kernel is then terminated. The DMR-RSM kernel follows a similar fault tolerance workflow, differing slightly in verification methods. It replicates the RSM module to keep successive computations within a specified tolerance range. The ABFT-GEMM kernel for **PV** is a row-wise tiling variant of kernel I.

3.2 End-to-End Protection for Attention

The operation-level protection framework introduces redundant memory access for tensors **S** and **P**, resulting in quadratic memory footprint and increased data transfer overhead, along with the need for three kernel launches per attention computation. Inspired by flash attention, we propose an end-to-end protection framework that achieves efficient fault tolerance in a single kernel.

• **End-to-End Protection Framework.** As shown in Figure 4, the end-to-end protection framework allocates a higher computational workload to each block, increasing arithmetic intensity. Each CTA is responsible for executing all computations required to generate the corresponding $\mathbf{O}_i$. Specifically, blocked ABFT-GEMM of $\mathbf{Q}_i\mathbf{K}_j^\top$ is computed iteratively for j ranging from 1 to n. The resulting $\mathbf{S}_{ij}$ values are subsequently processed using block softmax to compute the local normalization weights, with selective neuron value restriction applied to enhance error resilience. Finally, $\mathbf{P}_{ij}\mathbf{V}_j$ is accumulated into $\mathbf{O}_i$, with $\mathbf{O}_i$ rescaled by the updated rowsum at each iteration. This follows the same intra-block loop order as the computation of $\mathbf{Q}_i\mathbf{K}_j^\top$, sequentially accumulating the results into $\mathbf{O}_i$. The GEMM and rescale operations in the final step are collectively safeguarded by an integrated ABFT mechanism. In the practical implementation, the stabilized softmax function is employed, which subtracts the maximum value of each row during computation to prevent numerical overflow. This requires an additional rescaling operation each time, multiplying by the ratio of the new row maximum to the previous row maximum. This additional computation is a linear scaling of the original result and can still be protected using ABFT. A detailed explanation of this procedure is provided in Section 3.4.

For the block tiling, **Q**, **K**, and **V** are partitioned along the *seq_len* dimension with the same block size B. Since the computations of $\mathbf{Q}_i\mathbf{K}_j^\top$ and $\mathbf{P}_{ij}\mathbf{V}_j$ are executed sequentially in each block, the fused EFTA kernel sends the $seq_len \times num_head \times \left\lceil \frac{seq_len}{B} \right\rceil$ blocks to the GPU at each launch.

• **End-to-End Fault Tolerance Workflow.** The workload of end-to-end fault tolerance attention is presented in Figure 5. The figure illustrates the computation and fault-tolerance protection process corresponding to a single update step of $\mathbf{O}_i$, while simplifying the details of overlaps in the pipeline for clarity. The computation of block softmax is decomposed into two primary operations: the exponential function, denoted as EXP, and the row-wise summation, denoted as RS. Additionally, a rescaling operation, referred to as RSC, is applied before updating the accumulated results. The purpose of RSC is to ensure that the final output $\mathbf{O}_i$ is computed based on the global row-max and row-sum across all blocks. The NVR stands for neuron value restriction, which detects errors by ensuring computation results remain within the theoretical range

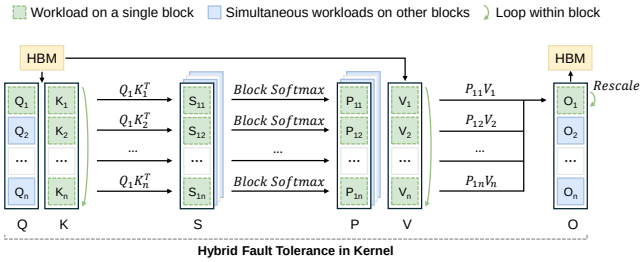


Figure 4: End-to-end fault tolerant attention at the block granularity.

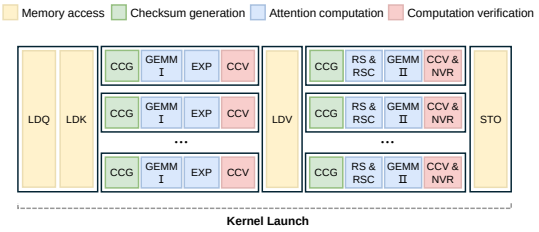


Figure 5: Workflow of the end-to-end fault tolerance with module-level protection.

of neuron outputs. This approach is widely used in deep learning [15, 47], providing lightweight protection.

In our end-to-end protection framework, we adopt an optimized selective NVR to impose different constraints based on the characteristics of each operation. Since the attention mechanism focuses on token positions with high similarity scores, it is crucial to maintain the relative magnitude between values. We employ ABFT to protect the GEMM of $\mathbf{Q}_i\mathbf{K}_j^\top$ and the EXP of $e^{\mathbf{S}_{ij}}$ using a unified set of checksums, ensuring precise protection for operations that determine relative numerical relationships. For the GEMM of $\mathbf{P}_{ij}\mathbf{V}_j$ and the RSC, a similar process is applied: the CCG generates input tensor checksums, which are computed alongside the original data, and the CCV verifies both operations simultaneously. For RS, which is used to compute the normalization factor and scale all values accordingly, we apply NVR with range-based protection. Moreover, by delaying detection, CCV and NVR are performed simultaneously, achieving a streamlined verification process with improved efficiency. In end-to-end attention, block softmax computes $\mathbf{P}_{ij}$ in place, reducing memory overhead and enabling long-sequence protection.

3.3 Strided ABFT Tailored for Tensor Core

In our end-to-end fault tolerant attention implementation, we employ the SM80_16×8×16_F32F16F16F32_TN MMA instruction for the GEMM operation, with its thread-to-data layout illustrated in Figure 6.

This instruction parallelizes execution across a warp of 32 threads, performing matrix multiplication on two half-precision (FP16) matrices with dimensions $M = 16$, $N = 8$, and $K = 16$, while accumulating the results in single precision (PF32). The matrix data are distributed across all threads within the warp, with each thread contributing a portion of the registers to collectively represent the matrix. As shown in Figure 6, each of the 32 threads (T_0 - T_{31}) provides

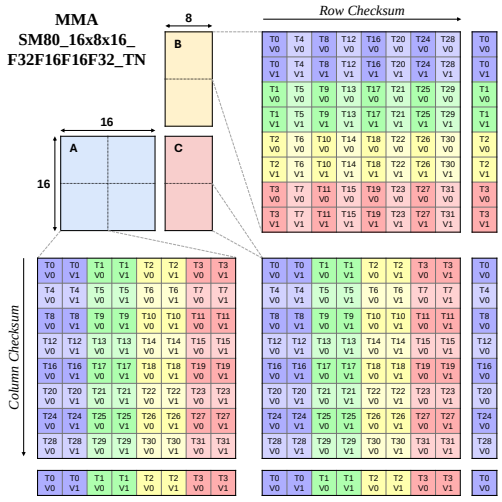


Figure 6: Thread-data layout of the SM80 16×8×16 F32 F16 F16 F32 TN MMA instruction with conventional checksum.

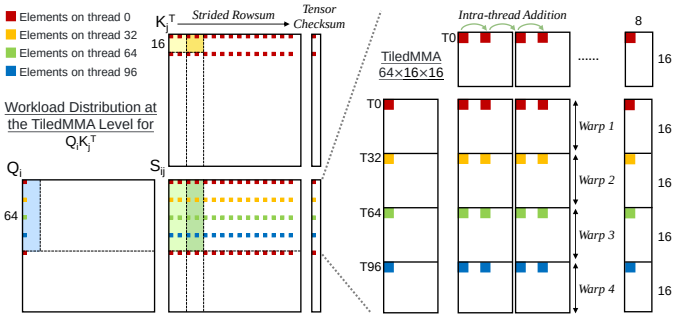


Figure 7: Workload distribution at the TiledMMA level for $Q_iK_j^T$, and the corresponding tensor checksum design.

two 2-byte registers V_0 and V_1, which together represent an 8×8 matrix tile. It can be observed that $\mathbf{A}[0][0]$ is stored in register V_0 of thread T_0, $\mathbf{A}[4][0]$ is stored in register V_0 of thread T_{16}, and $\mathbf{A}[8][0]$ is stored in register V_0 of thread T_0. The data corresponding to a single column is distributed across different threads, and this significantly reduces the efficiency of traditional column checksum computations of $\mathbf{A}^{c1} = \sum_{i=0}^{M-1} \mathbf{A}_{i,0:K-1}$ and $\mathbf{A}^{c2} = \sum_{i=0}^{M-1} (i+1)\mathbf{A}_{i,0:K-1}$. To compute the checksum, the data from each column must be aggregated and transferred to a single thread for accumulation, producing a $1 \times K$ tensor. This causes thread divergence and communication overhead, limiting the efficient utilization of the MMA instruction's minimal computational unit. The same issue also occurs during the computation of the row checksum. Therefore, a tensor checksum is proposed to address these issues.

• **Tensor Checksum Design.** The block level tensor checksum design is demonstrated in Figure 7. In this case, $\mathbf{Q}_i\mathbf{K}_j^T$ is used as an example. The computation of a block GEMM is further divided into smaller operations by TiledMMA. The TiledMMA is constructed by combining multiple MMA Atoms of PTX instructions, enabling more complex partitioning patterns. For the end-to-end fault tolerant attention, four warps (128 threads) work in parallel to compute a 64×16×16 part of $\mathbf{S}_{ij} = \mathbf{Q}_i\mathbf{K}_j^T$, and the full block computation is covered by repeating the TiledMMA operation along the M, N, and K dimensions. This 64×16×16 TiledMMA configuration increases warp-level parallelism along the M dimension, replicating the workload along the N dimension. As shown in Figure 7, along the column, elements with a stride of 64 are on the same thread. For instance, $\mathbf{Q}_i[0][0]$, $\mathbf{Q}_i[64][0]$, and $\mathbf{Q}_i[128][0]$ are stored in thread 0. Along the row, elements with a stride of 8 are on the same thread. The layout shows that $\mathbf{K}_j^\top[0][0]$, $\mathbf{K}_j^\top[0][8]$, and $\mathbf{K}_j^\top[0][16]$ are stored in thread 0. Based on this observation, the tensor checksum is designed by adding the row or column elements that are stored in the same thread at a fixed stride. The row-wise tensor checksum for any block of $\mathbf{K}^\top$ can be constructed as follows:

$$\text{tensor checksum}_1(\mathbf{K}^\top) = \sum_{l=0}^{lc} \mathbf{K}^\top_{:,l:l+s} \circ \mathbf{r}_1 \in \mathbb{R}^{d\times s} \tag{12}$$

$$\text{tensor checksum}_2(\mathbf{K}^\top) = \sum_{l=0}^{lc} \mathbf{K}^\top_{:,l:l+s} \circ \mathbf{r}_2 \in \mathbb{R}^{d\times s} \tag{13}$$

where lc represents the loop count of strided addition along the row, s denotes the chosen stride, and d is the head dimension of tensor $\mathbf{K}^\top$. The tensor checksum weights $\mathbf{r}_1$ and $\mathbf{r}_2$ have dimensions $d \times B$, where $\mathbf{r}_1$ is an all-one tensor, and $\mathbf{r}_2$ is filled with the value l+1. In our end-to-end fault tolerant attention, the row stride s is set to 8, since TiledMMA populates the N dimension by iteratively computing the 8×8 MMA Atom. Therefore, we can derive that $lc = \lceil d/8 \rceil - 1$. We can encode the column tensor checksum in a similar manner. Based on the analysis above, this column checksum must be computed with a stride of 64 and should have dimensions of $64 \times d$. This will incur 8 times memory overhead than the row tensor checksum. Therefore, for the ABFT scheme applied to attention, we adopt a row-checksum-only design. The same tensor checksum encoding method is also employed for fault tolerance in $\mathbf{P}_{ij}\mathbf{V}_j$ within the attention mechanism and can be extended to mixed-precision linear operations in the feed-forward layers.

• **Strided ABFT with Tensor Checksum.** With the proposed tensor checksum design, we implement an efficient algorithm-based fault tolerance mechanism for linear operations on Tensor Cores. For each block computing $\mathbf{QK}^\top$ in the end-to-end fault tolerance attention, two row-wise tensor checksums for $\mathbf{K}^\top$, denoted as $\mathbf{K}^\top_{check1}$ and $\mathbf{K}^\top_{check2}$, are encoded following Equations (12) and (13). These checksums are appended as new columns to the original tensors, and left multiplied with $\mathbf{Q}$, producing the corresponding row checksums $\mathbf{S}^\top_{check1}$ and $\mathbf{S}^\top_{check2}$:

$$\mathbf{S}_{check1} = \mathbf{QK}^\top_{check1} = \sum_{l=0}^{lc} \mathbf{QK}^\top_{:,l:l+s} \circ \mathbf{r}_1 = \sum_{l=0}^{lc} \mathbf{S}^\top_{:,l:l+s} \circ \mathbf{r}_1 \tag{14}$$

$$\mathbf{S}_{check2} = \mathbf{QK}^\top_{check2} = \sum_{l=0}^{lc} \mathbf{QK}^\top_{:,l:l+s} \circ \mathbf{r}_2 = \sum_{l=0}^{lc} \mathbf{S}^\top_{:,l:l+s} \circ \mathbf{r}_2 \tag{15}$$

where the both tensor checksum have dimensions $B_r \times s$. If no errors occur in the computation, the generated tensor checksum $\mathbf{S}_{check1}$ should maintain a strided addition relationship with tensor $\mathbf{S}$, where each element $\mathbf{S}_{check1}[i][j] = \sum_{l=0}^{lc} \mathbf{S}[i][j+s\times l]$. However, when computational errors arise, this equality no longer holds. In

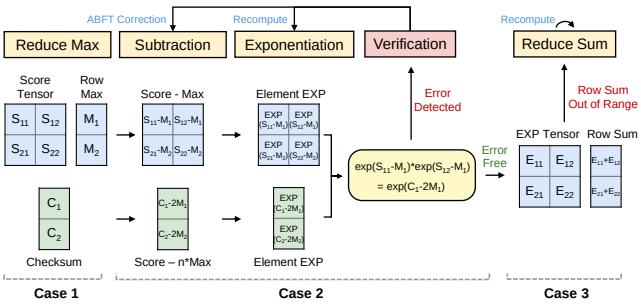


Figure 8: Selective neuron value restriction for softmax computation, covering all potential error scenarios.

this case, the error can be located in the row index i and column index $j + s \times \frac{\mathbf{S}_{check2}[i][j] - \mathbf{S}_{sum2}[i][j]}{\mathbf{S}_{check1}[i][j] - \mathbf{S}_{sum1}[i][j]}$ in the original tensor $\mathbf{S}$, where $\mathbf{S}_{sum1} = \sum_{l=0}^{lc} \mathbf{S}^{\top}_{:,l:l+s} \circ \mathbf{r}_1$ and $\mathbf{S}_{sum2} = \sum_{l=0}^{lc} \mathbf{S}^{\top}_{:,l:l+s} \circ \mathbf{r}_2$. The error can be corrected by adding $\mathbf{S}_{check1}[i][j] - \mathbf{S}_{sum1}[i][j]$ to the corresponding element. Since our tensor checksum employs an interleaved encoding scheme with additional checksum positions, the strided ABFT exhibits stronger error detection and correction capabilities compared to single-row or single-column ABFT. Specifically, for the row tensor checksum utilized in our end-to-end fault tolerant attention, errors within each row can be detected and corrected as long as their positions are not spaced at multiples of 8. Theoretically, this approach can enhance fault tolerance by up to a factor of 8 compared to traditional ABFT.

3.4 Selective Neuron Value Restriction on Softmax

In softmax computations, the exponential operation determines the relative magnitudes of the output values, playing a crucial role in establishing inter-token dependency in the attention mechanism. The normalization step ensures that the results remain within a stable numerical range, which can be protected using range-based approaches. The proposed selective neuron value restriction (SNVR) method leverages the invariance of computational relationships to apply precise protection to exponential operations, while constraining normalization results within the theoretical range. This approach enables fine-grained fault tolerance in softmax computations, balancing reliability and efficiency. The SNVR applies fault tolerant mechanisms tailored to each possible error scenario, under the assumption that different types of fault do not occur simultaneously. Additionally, to mitigate numerical overflow, this work employs the numerically stable softmax formulation: $p_{i,j} = \frac{\exp(s_{i,j} - s_{i_max})}{\sum_k \exp(s_{i,k} - s_{i_max})}$, where $s_{i,j}$ denotes an element of the tensor $\mathbf{S}$, and s_{i_max} represents the maximum value in the i-th row. Figure 8 illustrates the fault tolerance mechanism of selective neuron value restriction under different computational errors using a simplified checksum. Below, We detail the fault tolerance process for each case based on the actual tensor checksums.

• **Case 1**: An error occurs during the search for the maximum row value. If the error occurs in row i, the rowmax s_{i_max} is altered to s'_{i_max}. During the computation of the numerator, this error propagates across the entire row, corrupting $\exp(s_{i,j} - s'_{i_max})$. Similarly, in the computation of the denominator, the error also contaminates the respective value $\sum_k \exp(s_{i,k} - s'_{i_max})$. However, such errors would not influence the final result, as the error terms $e^{s'_{i_max}}$ are canceled out.

• **Case 2**: The calculation of $d_{i,j} = s_{i,j} - s_{i_max}$ is affected by a computational fault or the exponential computation $\exp(d_{i,j})$ is erroneous. In this case, the result of these two steps is corrupted to $(e^{s_{i,j} - s_{i_max}})'$. This one-point error would further propagate to the i-th row of the GEMM computation $e^{(\mathbf{S} - \mathbf{S}_{max})}V$ and the i-th rowsum value $\sum_k \exp(s_{i,k} - s_{i_max})$. Therefore, this error need be corrected before these subsequent computations. The strided ABFT can be extended to protect the max value deduction and the exponentiation by reusing the checksum when calculating the tensor $\mathbf{S}$. When we conduct element-wise deduction, the same operation is applied on the checksum, and the equality between the checksum and original data becomes $\mathbf{S}_{check1}[i][j] - (lc+1)s_{i_max} = \sum_{l=0}^{lc} \mathbf{S}[i][j + s \times l] - s_{i_max}$. Similarly, after the exponential computation, the checksum can be verified using $\exp(\mathbf{S}_{check1}[i][j] - (lc+1)s_{i_max}) = \prod_{l=0}^{lc} \exp(\mathbf{S}[i][j + s \times l] - s_{i_max})$. If this equation is not satisfied in the verification process, it indicates the occurrence of a computational error. For error correction, if the error arises in the linear operations of deduction, it can be directly corrected using checksum-based recovery; if the error occurs in exponentiation, we employ recomputation for correction.

• **Case 3**: When computing the denominator of the softmax function, errors may occur during the reduce sum operation, causing the i-th rowsum to be incorrectly computed as $\sum'_k \exp(s_{i,k} - s_{i_max})$. This leads to errors in all elements of the i-th row during the final computation of $\mathbf{P}$, as the normalization process scales the entire row by an incorrect factor. However, since this error does not affect the relative numerical relationships, the attention mechanism still focuses on the positions with the highest values. Therefore, it is sufficient to ensure that these values remain within the theoretical computational range, preventing significant deviations or anomalies. In practice, we constrain the rowsum within a predefined range using the following inequality: $\sum \exp(s_{block_max} - s_{global_max}) < rowsum < n$. If the computed rowsum falls outside this range, it indicates a computational error, which is then corrected through recomputation.

• **Optimized EFTA with Unified Verification.** To further reduce the overhead of fault tolerance, a checksum reuse strategy is adopted to keep track of the invariant numerical relationship across multiple computations, allowing error detection through a single verification step, as presented in Algorithm 1. The tensor checksum $\mathbf{S}^{c_1}_{ij}$ generated in GEMM I is subsequently processed through subtraction and element-wise EXP operations, integrating the information of the three computations into a unified checksum. Errors can be detected by verifying the equality of element-wise products between the original computation result $\mathbf{P}_{ij}$ and the corresponding tensor checksum $\mathbf{P}^{c_1}_{ij}$. Since the tensor $\mathbf{P}_{ij}$ is used in place during the GEMM II computation, error verification must be performed at each iteration to prevent error propagation in subsequent calculations. For GEMM II, rescale, and normalization operations, we adopt a similar unified verification strategy. Using the linearity of these three computations, we apply the same transformations to the tensor checksum $\mathbf{O}^{c_1}_i$, ensuring that its element-wise summation remains consistent with that of the original tensor $\mathbf{O}_i$. As

Algorithm 1 EFTA with Unified Verification

Tile $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ along seq_len dimension into $blk = \left\lceil \frac{seq_len}{B} \right\rceil$ blocks,
where $\mathbf{Q}_i, \mathbf{K}_j, \mathbf{V}_j \in \mathbb{R}^{B \times d}$
for i from 1 to blk **do**
  Load $\mathbf{Q}_i$ from HBM to Tensor Core shared memory
  for j from 1 to blk **do**
    Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to Tensor Core shared memory
    Encode tensor checksum $\mathbf{K}_j^{c_1}, \mathbf{K}_j^{c_2}, \mathbf{V}_j^{c_1}, \mathbf{V}_j^{c_2}$
    GEMM I: $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T$, $\mathbf{S}_{ij}^{c_1} = \mathbf{Q}_i (\mathbf{K}_j^{c_1})^T$, $\mathbf{S}_{ij}^{c_2} = \mathbf{Q}_i (\mathbf{K}_j^{c_2})^T$
    Reduce Max: $m_{ij} = \max(m_{i(j-1)}, \text{rowmax}(\mathbf{S}_{ij}))$
    Element-wise EXP: $\mathbf{P}_{ij} = \exp(\mathbf{S}_{ij} - m_{ij})$
      $\mathbf{P}_{ij}^{c_1} = \exp(\mathbf{S}_{ij}^{c_1} - \lceil \frac{d}{8} \rceil m_{ij})$
    if $|\prod \mathbf{P}_{ij} - \prod \mathbf{P}_{ij}^{c_1}| > \epsilon_1$ **then**
      Correct linear operations with tensor checksum $\mathbf{S}_{ij}^{c_1}, \mathbf{S}_{ij}^{c_2}$
      Correct EXP with recomputation
    end if
    Reduce Sum: $\ell_{ij} = e^{m_{i(j-1)} - m_{ij}} \ell_{i(j-1)} + \text{rowsum}(\mathbf{P}_{ij})$
    GEMM II: $\mathbf{O}_i = \text{diag}(e^{m_{i(j-1)} - m_{ij}}) \mathbf{O}_i + \mathbf{P}_{ij} \mathbf{V}_j$
      $\mathbf{O}_i^{c_1} = \text{diag}(e^{m_{i(j-1)} - m_{ij}}) \mathbf{O}_i^{c_1} + \mathbf{P}_{ij} \mathbf{V}_j^{c_1}$
      $\mathbf{O}_i^{c_2} = \text{diag}(e^{m_{i(j-1)} - m_{ij}}) \mathbf{O}_i^{c_2} + \mathbf{P}_{ij} \mathbf{V}_j^{c_2}$
  end for
  if $\ell_{ij} < \sum_k e^{(m_{ik} - m_{ij})}$ or $\ell_{ij} > seq_len$ **then**
    $\ell_{ij} = \sum_k e^{(m_{ik} - m_{ij})}$
  end if
  Normalization: $\mathbf{O}_i = \text{diag}(\ell_{ij})^{-1} \mathbf{O}_i$, $\mathbf{O}_i^{c_1} = \text{diag}(\ell_{ij})^{-1} \mathbf{O}_i^{c_1}$
    $\mathbf{O}_i^{c_2} = \text{diag}(\ell_{ij})^{-1} \mathbf{O}_i^{c_2}$
  if $|\sum \mathbf{O}_i - \sum \mathbf{O}_i^{c_1}| > \epsilon_2$ **then**
    Correct linear operations with tensor checksum $\mathbf{O}_i^{c_1}, \mathbf{O}_i^{c_2}$
  end if
end for

the final result $\mathbf{O}_i$ is not fused with subsequent computations, the numerical relationship between the tensor checksum $\mathbf{O}_i^{c_1}$ and $\mathbf{Q}_i$ can be preserved throughout updates. This allows verification to be performed only once, after all iterations have been completed.

We also optimize the error detection process for the reduce sum operation. Rather than verifying the range of rowsum ℓ_{ij} after each reduce sum operation, we instead validate the final value of ℓ_{ij} prior to normalization, ensuring that it remains within the theoretically expected bounds. Moreover, we correct errors in rowsum by replacing them with the approximation result of the normalization factor, thereby mitigating the overhead of recomputation. In practical implementation, this approximation value is derived from the computation of $\sum_k e^{(m_{ik} - m_{ij})}$, where m_{ik} is the maximum row value in each iteration and m_{ij} is the global rowmax. This approximate correction still ensures reliable inference, as attention primarily focuses on the most important positions, with most values concentrated around the largest ones, while the rest have a negligible impact on the summation.

4 Experimental Evaluation

We evaluate our end-to-end fault tolerant attention (EFTA) on an NVIDIA 40GB A100-PCIE GPU. The GPU device is connected to a node with one 64-core AMD EPYC 7763 CPU whose boost frequency is 3.5 GHz. We compile programs using CUDA 12.4 with optimization flag O3. The input and output tensors for EFTA are

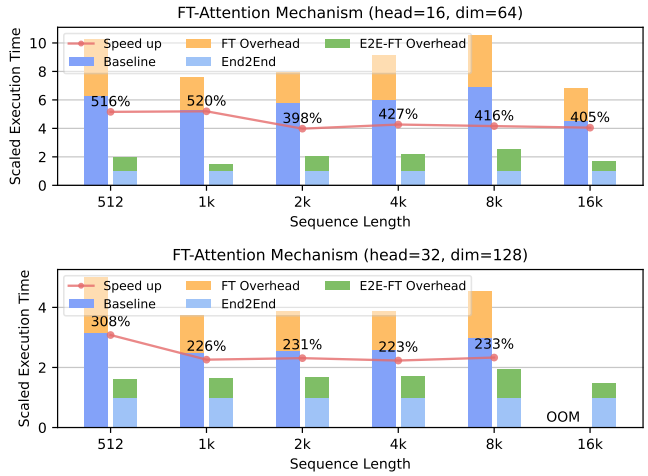


Figure 9: Scaled execution time and fault tolerance overhead of end-to-end FT attention and operation-level FT attention.

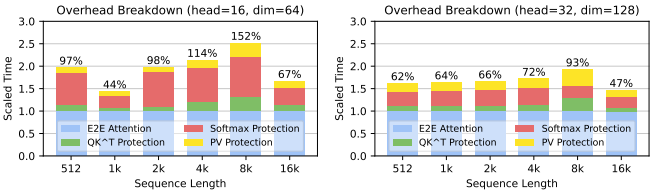


Figure 10: Breakdown analysis of fault tolerance overhead in the end-to-end FT attention.

in half-precision floating-point format (FP16), and SM80 16×8×16 F32F16F16F32 TN MMA instruction is utilized for mixed-precision matrix computations on the tensor core. We first present benchmark results comparing EFTA with existing operation-level protection approaches [25, 27, 46] in different attention configurations. The baseline is formed by integrating these methods into the fault-tolerance framework presented in Section 3.1. Next, we assess the fault tolerance overhead and the error coverage of strided ABFT and SNVR respectively. Finally, we simulate the fault tolerance performance of the optimized EFTA on common Transformer models.

4.1 End-to-End Fault Tolerance Framework for Attention Mechanism

In this subsection, we measure the runtime of EFTA and existing operation-level fault tolerance method and conduct an overhead breakdown analysis. We compare their performance across sequence lengths ranging from 512 to 16K, adjusting the batch size to maintain a total token count of 16K. Our experiments evaluate attention performance and the corresponding fault tolerance overhead under two hidden dimension settings. Based on common configurations of medium and large models, the hidden dimension of attention is set to 1024 (head = 16, dim = 64) and 4096 (head = 32, dim = 128).

• **End-to-End Fault Tolerance Performance.** Figure 9 shows the performance of the end-to-end fault tolerance framework compared to the operation-level fault tolerance framework in attention computation. For attention computation in the medium model with

hidden dimension = 1024, our framework achieves an average 447% speedup over the traditional method under fault tolerance protection. Notably, for short inputs with sequence lengths of 512 and 1k, the speedup exceeds fivefold. For the large-scale model configuration with hidden dimension = 8192, our framework achieves an average 244% speedup, and when sequence length = 512, the speedup can reach threefold. Additionally, it can be observed that the traditional method encounters an out-of-memory (OOM) error when the sequence length reaches 16k. This occurs because the traditional approach stores the intermediate results of the $\mathbf{QV}^\top$ matrix computation, requiring $batch \times num_head \times seq_len^2$ memory access to HBM. Our end-to-end method mitigates this issue by partitioning the intermediate data and incrementally accumulating the results, ensuring fault tolerance in long-sequence inference.

• **FT Overhead Breakdown Analysis.** The detailed fault tolerance overhead breakdown analysis of EFTA is provided in Figure 10.

The overhead analysis indicates that although our EFTA achieves a significant speedup over existing approaches, the overhead remains significant. For the attention mechanism of large-scale models, EFTA incurs an average fault tolerance overhead of 68%. For medium-scale models, the average overhead is 96%. Such high overhead stems from the incompatibility of traditional methods with EFTA's workflow and computing platform. EFTA optimizes the computation pipeline by parallelizing operations across phases, but DMR's redundant computations cannot be efficiently integrated into the fused kernel, requiring a separate phase for fault tolerance and causing 47% average overhead for softmax protection. Meanwhile, traditional ABFT, designed for single and double precision computations, lacks efficient optimizations for mixed-precision computations on Tensor Core, leading to 35% average overhead for GEMM protection. Therefore, we develop strided ABFT to reduce the fault tolerance overhead of linear computations and apply SNVR to enhance error detection efficiency for nonlinear computations.

4.2 EFTA with Strided ABFT

The same experimental setup is adopted as in Section 4.1. We evaluate the fault tolerance overhead of strided ABFT under different hidden dimension settings and assess its error coverage capability. For each attention computation cycle, we inject one bit-flip to a random bit position of a random value, and record the fault detection rate and alarm rate. During this process, we gradually increased the error threshold to reduce false positives while still capturing as many injected errors as possible.

• **Fault Tolerance Overhead of Strided ABFT.** Figure 11 compares the performance of EFTA using strided ABFT and traditional ABFT. The execution time for these two ABFT methods is represented by the purple and orange bars respectively, and the overhead is calculated by $\frac{fault\ tolerance\ time}{attention\ execution\ time}$ and is displayed at the top of the bars. In the experiment, the ABFT is used to protect $\mathbf{QK}^\top$ and $\mathbf{PV}$ computations, and results show that with head = 16, dim = 64, and a total input token count of 16K, strided ABFT incurs an average overhead of 11.8% in EFTA. This reduces approximately 64% fault tolerance overhead compared to traditional ABFT. The line plot in the figure illustrates the relative overhead of the two methods. It shows that for sequence lengths ranging from 512 to

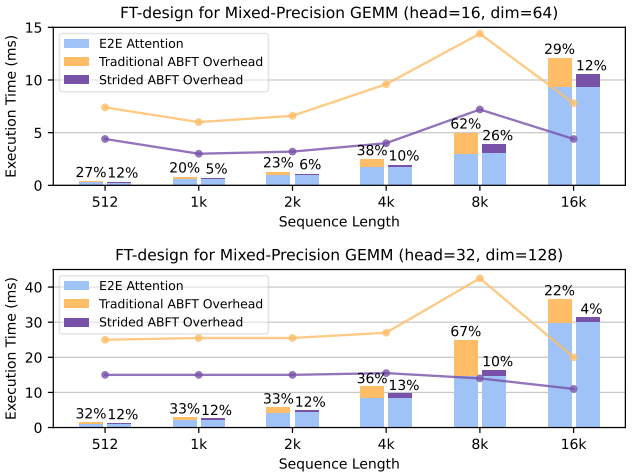


Figure 11: End-to-end FT attention execution time: comparison of strided ABFT and traditional ABFT.

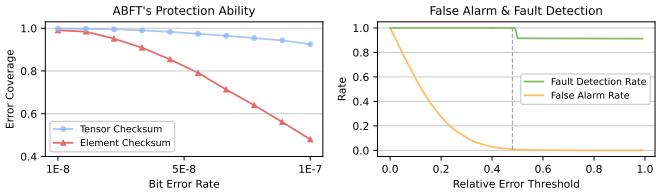


Figure 12: Error coverage and false alarm rate analysis of strided ABFT.

16K, strided ABFT consistently achieves lower fault tolerance overhead. For head = 32, dim = 128, and a total input token count of 16K, the average fault tolerance overhead of strided ABFT is 10.5%, which still outperforms the traditional approach across all the test sequence lengths.

• **Error Coverage of strided ABFT.** In Figure 12, the left plot illustrates the error detection capability of traditional element checksum-based ABFT and our strided ABFT. The right plot presents the true detection rate and the false detection rate of strided ABFT under different error threshold settings. strided ABFT is optimized according to the data-thread layout of tensor cores, employing an 8-element-wide checksum design that aligns with MMA instructions. This approach improves error coverage compared to the single-element-wide checksum used in traditional ABFT. When the computational bit error rate is 1e-7, strided ABFT achieves an error coverage rate of 92.5%, whereas traditional checksum achieves only 48%. Due to intrinsic rounding errors of half-precision tensor core computations and ABFT's summation-based detection, checksum results may differ from original outputs under error-free conditions. Therefore, in practical error detection, a true error is identified when the difference between the checksums and original results exceeds the error threshold. Our experiments show that when the threshold is below 0.4, intrinsic errors are misidentified as true errors, while a threshold above 0.5 leads to some computational errors being missed. The error threshold of 0.48 gives the best detection performance. For multiple errors occurring in one computation cycle, our method can still cover in relatively low overhead, as it

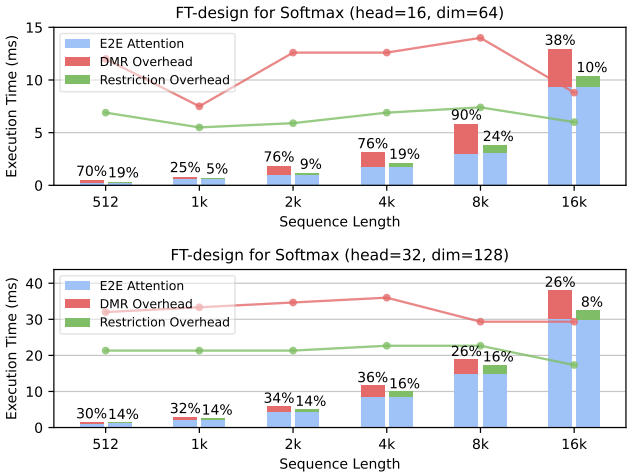


Figure 13: End-to-end FT attention execution time: comparison of DMR protection and selective neuron value restriction.

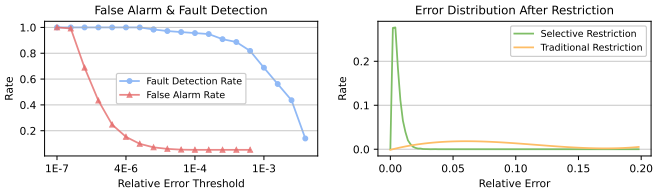


Figure 14: False alarm rate and error distribution analysis of selective neuron value restriction.

partitions the sequence dimension into parallel blocks, each with independent fault tolerance. Errors in multiple operations likely occur in different blocks and can be corrected concurrently, resulting in a minimal overhead increase. In rare cases where multiple errors occur within the same block, this requires two correction stages for fault tolerance, resulting in at most 2× overhead.

4.3 EFTA with Selective Neuron Value Restriction

• **Fault Tolerance Overhead of SNVR.** Within the EFTA framework, we employ DMR and selective neuron value restriction separately to protect softmax computation and compare their performance. We measure the total execution time and the fault tolerance time for both methods and derive the corresponding overhead. The experimental results are presented in Figure 13. Medium-scale models typically use attention with 1024 hidden dimensions. In this setting, selective neuron value restriction incurs an average fault tolerance overhead of 14.3%, whereas DMR results in a significantly higher overhead of 62.5%. For large model settings with a hidden dimension of 4096, selective neuron value restriction incurs an average overhead of 13.6%, reducing overhead by half compared to DMR's 30.6%. The relative overhead line in the figure shows that our selective neuron value restriction achieves better fault tolerance performance for all test sequence length.

• **Error Coverage of Selective Neuron Value Restriction.** We further explore the SNVR method's fault detection and false alarm rates under different relative error thresholds. The results

Table 2: Comparison of EFTA and optimized EFTA for head = 16, dim = 64

Length	EFTA (ms)	Overhead	EFTA-o (ms)	Overhead
512	0.425	52.3%	0.315	12.9%
1k	0.924	40.2%	0.718	8.9%
2k	1.537	48.0%	1.178	13.4%
4k	2.924	66.5%	2.004	14.1%
8k	4.966	62.9%	3.951	29.6%
16k	13.804	48.2%	10.507	12.8%

Table 3: Comparison of EFTA and optimized EFTA for head = 32, dim = 128

Length	EFTA (ms)	Overhead	EFTA-o (ms)	Overhead
512	1.498	24.9%	1.199	13.4%
1k	2.810	24.7%	2.253	13.5%
2k	5.441	24.6%	4.364	13.4%
4k	10.703	26.1%	8.483	14.8%
8k	18.912	27.0%	14.886	15.4%
16k	32.728	9.1%	29.995	4.5%

are shown in the left plot of Figure 14. By analyzing the numerical difference between the checksum and the actual result in both error-free and error-injected scenarios, we found that when the relative error threshold is set to 7e-6, the fault detection rate is around 97.2% and the false alarm rate is 5.9%, maximizing the error coverage. Additionally, SNVR protects the numerator and denominator separately in softmax computation, while the traditional restriction method only safeguards the final division result. This gives SNVR a higher error coverage rate and a lower correction error. The error distribution after applying restriction methods is plotted on the right side of Figure 14. The results show that the SNVR method restricts most errors within the range of 0 to 0.02. In contrast, traditional methods cannot confine errors within a small range, with values widely distributed between 0 and 0.15, leading to greater uncertainty in fault tolerance.

4.4 Optimized EFTA and Fault Tolerance in Transformers

In this subsection, we first evaluate the total execution time and fault tolerance overhead of the optimized end-to-end fault tolerant attention with unified verification. We then simulate the overhead of error detection and correction using the optimized EFTA on common Transformer models.

• **EFTA with Unified Verification.** We further optimized the EFTA using a unified verification strategy, encoding multi-step computations into a single tensor checksum for error detection. As shown in Table 2, when 16 attention heads are used and the head dimension is set to 64, the optimized EFTA reduces the average fault tolerance overhead to 15.3% compared to 53% in the unoptimized version, achieving a 1.32× speedup on average. Furthermore, compared to the operation-level protection method, optimized EFTA achieves an average speedup of 7.56×.

For the attention setting of large models with head = 32 and dim = 64, the optimized EFTA incurs an average overhead of 12.5% for fault tolerance, compared to 22.7% in the unoptimized version.

Table 4: Model configurations for fault tolerance analysis

Model	Attention Layer	Head Count	Head Dimension
GPT2	12	12	64
BERT-Base	12	12	64
BERT-Large	24	16	64
T5-Small	18	8	64

Our method is on average 3.69× faster than existing operation-level fault tolerance approaches. The corresponding results are presented in Table 3.

• **EFTA on Transformer Models.** To further investigate the fault tolerance performance of our end-to-end fault-tolerant attention in practical applications, we simulate the error detection and correction overhead of applying optimized EFTA on common transformer models, including GPT2, BERT-Base, BERT-Large, and T5-Small. The corresponding model configurations are shown in Table 4. During the experiments, we fixed the input length at 512 and set the hidden dimension and layer counts of EFTA according to the model configuration. We profile the execution time for each inference step and the simulation results show that the GPT2 model takes approximately 5.6 ms to generate one token. This time increases to 5.874 ms when EFTA is used. Figure 15 provides a detailed breakdown of execution time and overhead. For error detection, EFTA incurs an average overhead of 4.7% across the four models. For error correction, we simulate errors by injecting a single bit flip for each attention computation, and correcting these errors incurs an average overhead of 9.1%.

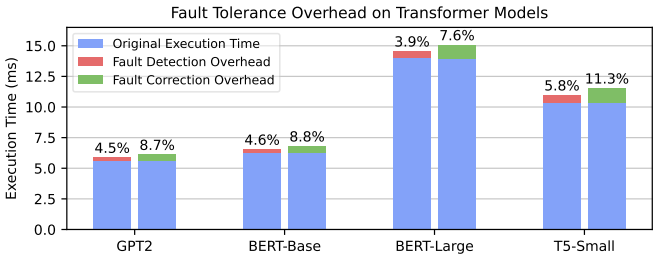


Figure 15: Error detection and correction overhead of EFTA on common Transformer models.

5 Conclusion

In this paper, we propose the FT-Transformer framework with end-to-end fault tolerant attention (EFTA) to enhance the reliability of Transformer model inference against soft errors. The proposed EFTA fuses attention and fault tolerance computations within a single kernel, mitigating redundant memory access. In addition, an architecture-aware and computation-optimized hybrid fault tolerance scheme is incorporated in EFTA, enabling efficient error detection and correction for mixed-precision matrix operations and softmax computations on Tensor Cores. Experimental results show that our EFTA achieves up to 7.56× speedup compared to existing operation-level protection methods with a 13.9% average fault tolerance overhead. Further simulations on Transformer models show that EFTA adds negligible overhead to the inference process - 4.7% for error detection and 9.1% for error correction.

References

[1] R.C. Baumann. 2005. Radiation-induced soft errors in advanced semiconductor technologies. *IEEE Transactions on Device and Materials Reliability* 5, 3 (2005), 305–316. https://doi.org/10.1109/TDMR.2005.853449

[2] Shekhar Borkar. 2005. Designing reliable systems from unreliable components: the challenges of transistor variability and degradation. *Ieee Micro* 25, 6 (2005), 10–16.

[3] Cristiano Castelfranchi. 1998. Modelling social action for AI agents. *Artificial intelligence* 103, 1-2 (1998), 157–182.

[4] Ching-Tai Chin, K. Mehrotra, C.K. Mohan, and S. Rankat. 1994. Training techniques to obtain fault-tolerant neural networks. In *Proceedings of IEEE 24th International Symposium on Fault- Tolerant Computing*. 360–369. https://doi.org/10.1109/FTCS.1994.315624

[5] C.-T. Chiu, K. Mehrotra, C.K. Mohan, and S. Ranka. 1993. Robustness of feed-forward neural networks. In *IEEE International Conference on Neural Networks*. 783–788 vol.2. https://doi.org/10.1109/ICNN.1993.298655

[6] L.-C. Chu and B.W. Wah. 1990. Fault tolerant neural networks with hybrid redundancy. In *1990 IJCNN International Joint Conference on Neural Networks*. 639–649 vol.2. https://doi.org/10.1109/IJCNN.1990.137773

[7] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=mZn2Xyh9Ec

[8] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems* 35 (2022), 16344–16359.

[9] Jacob Devlin. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).

[10] Fernando Morgado Dias and Ana Antunes. 2008. Fault tolerance improvement through architecture change in artificial neural networks. In *International Symposium on Intelligence Computation and Applications*. Springer, 248–257.

[11] Vasisht Duddu, D Vijay Rao, and Valentina Balas. 2020. Towards enhancing fault tolerance in neural networks. In *MobiQuitous 2020-17th EAI International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services*. 59–68.

[12] M.D. Emmerson and R.I. Damper. 1993. Determining and improving the fault tolerance of multilayer perceptrons in a pattern-recognition application. *IEEE Transactions on Neural Networks* 4, 5 (1993), 788–793. https://doi.org/10.1109/72.248456

[13] Graham E Fagg and Jack J Dongarra. 2000. FT-MPI: Fault tolerant MPI, supporting dynamic applications in a dynamic world. In *European parallel virtual machine/message passing interface users' group meeting*. Springer, 346–353.

[14] David Freitas, César Marcon, Jarbas Silveira, Lirida Naviner, and João Mota. 2022. A survey on two-dimensional Error Correction Codes applied to fault-tolerant systems. *Microelectronics Reliability* 139 (2022), 114826.

[15] Giulio Gambardella, Johannes Kappauf, Michaela Blott, Christoph Doehring, Martin Kumm, Peter Zipf, and Kees Vissers. 2019. Efficient error-tolerant quantized neural network accelerators. In *2019 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT)*. IEEE, 1–6.

[16] Zhen Gao, Ziye Yin, Jingyan Wang, Rui Su, Jie Deng, Qiang Liu, Pedro Reviriego, Shanshan Liu, and Fabrizio Lombardi. 2025. On the Dependability of Bidirectional Encoder Representations from Transformers (BERT) to Soft Errors. *IEEE Transactions on Nanotechnology* 24 (2025), 73–87. https://doi.org/10.1109/TNANO.2025.3531721

[17] Behnam Ghavami, Mani Sadati, Zhenman Fang, and Lesley Shannon. 2022. FitAct: Error Resilient Deep Neural Networks via Fine-Grained Post-Trainable Activation Functions. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 1239–1244. https://doi.org/10.23919/DATE54114.2022.9774635

[18] Anthony Gillioz, Jacky Casas, Elena Mugellini, and Omar Abou Khaled. 2020. Overview of the Transformer-based Models for NLP Tasks. In *2020 15th Conference on computer science and information systems (FedCSIS)*. IEEE, 179–183.

[19] Ghouthi Boukli Hacene, François Leduc-Primeau, Amal Ben Soussia, Vincent Gripon, and François Gagnon. 2019. Training Modern Deep Neural Networks for Memory-Fault Robustness. In *2019 IEEE International Symposium on Circuits and Systems (ISCAS)*. 1–5. https://doi.org/10.1109/ISCAS.2019.8702382

[20] Kai Han, Yunhe Wang, Hanting Chen, Xinghao Chen, Jianyuan Guo, Zhenhua Liu, Yehui Tang, An Xiao, Chunjing Xu, Yixing Xu, et al. 2022. A survey on vision transformer. *IEEE transactions on pattern analysis and machine intelligence* 45, 1 (2022), 87–110.

[21] Kuang-Hua Huang and Jacob A. Abraham. 1984. Algorithm-Based Fault Tolerance for Matrix Operations. *IEEE Trans. Comput.* C-33, 6 (1984), 518–528. https://doi.org/10.1109/TC.1984.1676475

[22] Younis Ibrahim, Haibin Wang, Junyang Liu, Jinghe Wei, Li Chen, Paolo Rech, Khalid Adam, and Gang Guo. 2020. Soft errors in DNN accelerators: A comprehensive review. *Microelectronics Reliability* 115 (2020), 113969.

[23] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. 2020. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*. PMLR, 5156–5165.

[24] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. 2020. Reformer: The Efficient Transformer. In *International Conference on Learning Representations*. https://openreview.net/forum?id=rkgNKkHtvB

[25] Jack Kosaian and K. V. Rashmi. 2021. Arithmetic-intensity-guided fault tolerance for neural network inference on GPUs. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (St. Louis, Missouri) *(SC '21)*. Association for Computing Machinery, New York, NY, USA, Article 79, 15 pages. https://doi.org/10.1145/3458817.3476184

[26] Hongliang Li, Hairui Zhao, Ting Sun, Xiang Li, Haixiao Xu, and Keqin Li. 2024. Interference-aware opportunistic job placement for shared distributed deep learning clusters. *J. Parallel and Distrib. Comput.* 183 (2024), 104776.

[27] Yuhang Liang, Xinyi Li, Jie Ren, Ang Li, Bo Fang, and Jieyang Chen. 2025. ATTNChecker: Highly-Optimized Fault Tolerant Attention for Large Language Model Training. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming* (Las Vegas, NV, USA) *(PPoPP '25)*. Association for Computing Machinery, New York, NY, USA, 252–266. https://doi.org/10.1145/3710848.3710870

[28] Tao Liu, Wujie Wen, Lei Jiang, Yanzhi Wang, Chengmo Yang, and Gang Quan. 2019. A Fault-Tolerant Neural Network Architecture. In *2019 56th ACM/IEEE Design Automation Conference (DAC)*. 1–6.

[29] Yinhan Liu. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* 364 (2019).

[30] Amir Moosavie Nia and Karim Mohammadi. 2007. A generalized ABFT technique using a fault tolerant neural network. *Journal of Circuits, Systems, and Computers* 16, 03 (2007), 337–356.

[31] Bin Nie, Devesh Tiwari, Saurabh Gupta, Evgenia Smirni, and James H. Rogers. 2016. A large-scale study of soft-errors on GPUs in the field. In *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 519–530. https://doi.org/10.1109/HPCA.2016.7446091

[32] E. L. Petersen, R. Koga, M. A. Shoga, J. C. Pickel, and W. E. Price. 2013. The Single Event Revolution. *IEEE Transactions on Nuclear Science* 60, 3 (2013), 1824–1835. https://doi.org/10.1109/TNS.2013.2248065

[33] D.S. Phatak and I. Koren. 1995. Complete and partial fault tolerance of feedforward neural nets. *IEEE Transactions on Neural Networks* 6, 2 (1995), 446–456. https://doi.org/10.1109/72.363479

[34] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.

[35] Brandon Reagen, Udit Gupta, Lillian Pentecost, Paul Whatmough, Sae Kyu Lee, Niamh Mulholland, David Brooks, and Gu-Yeon Wei. 2018. Ares: A framework for quantifying the resilience of deep neural networks. In *2018 55th ACM/ESDA/IEEE Design Automation Conference (DAC)*. 1–6. https://doi.org/10.1109/DAC.2018.8465834

[36] G.A. Reis, J. Chang, N. Vachharajani, R. Rangan, and D.I. August. 2005. SWIFT: software implemented fault tolerance. In *International Symposium on Code Generation and Optimization*. 243–254. https://doi.org/10.1109/CGO.2005.34

[37] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. 2021. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics* 9 (2021), 53–68.

[38] J. Sosnowski. 1989. Fault Tolerant DMR Miroprocessor System. *IFAC Proceedings Volumes* 22, 19 (1989), 141–146. https://doi.org/10.1016/S1474-6670(17)52809-8 IFAC/IFIP Workshop on Safety of Computer Control Systems 1989 (SAFECOMP '89), Vienna, Austria, 5-7 December.

[39] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023).

[40] A Vaswani. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* (2017).

[41] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. 2020. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768* (2020).

[42] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.

[43] Shixun Wu, Yitong Ding, Yujia Zhai, Jinyang Liu, Jiajun Huang, Zizhe Jian, Huangliang Dai, Sheng Di, Bryan M Wong, Zizhong Chen, et al. 2024. FT K-Means: A High-Performance K-Means on GPU with Fault Tolerance. In *2024 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 322–334.

[44] Shixun Wu, Yujia Zhai, Jinyang Liu, Jiajun Huang, Zizhe Jian, Huangliang Dai, Sheng Di, Zizhong Chen, and Franck Cappello. 2024. Turbofft: A high-performance fast fourier transform with fault tolerance on gpu. *arXiv preprint arXiv:2405.02520* (2024).

[45] Peng Xu, Xiatian Zhu, and David A Clifton. 2023. Multimodal learning with transformers: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 10 (2023), 12113–12132.

[46] Xinghua Xue, Cheng Liu, Ying Wang, Bing Yang, Tao Luo, Lei Zhang, Huawei Li, and Xiaowei Li. 2023. Soft Error Reliability Analysis of Vision Transformers. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 31, 12 (2023), 2126–2136. https://doi.org/10.1109/TVLSI.2023.3317138

[47] Jinyu Zhan, Ruoxu Sun, Wei Jiang, Yucheng Jiang, Xunzhao Yin, and Cheng Zhuo. 2022. Improving Fault Tolerance for Reliable DNN Using Boundary-Aware Activation. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 10 (2022), 3414–3425. https://doi.org/10.1109/TCAD.2021.3129114

[48] Hairui Zhao, Hongliang Li, Qi Tian, Jie Wu, Meng Zhang, Zhewen Xu, Xiang Li, and Haixiao Xu. 2025. ArrayPipe: Introducing Job-Array Pipeline Parallelism for High Throughput Model Exploration. In *IEEE INFOCOM 2025-IEEE Conference on Computer Communications*. IEEE, 1–10.

[49] Hairui Zhao, Qi Tian, Hongliang Li, and Zizhong Chen. 2025. {FlexPipe}: Maximizing Training Efficiency for Transformer-based Models with {Variable-Length} Inputs. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. 143–159.

[50] Kai Zhao, Sheng Di, Sihuan Li, Xin Liang, Yujia Zhai, Jieyang Chen, Kaiming Ouyang, Franck Cappello, and Zizhong Chen. 2020. FT-CNN: Algorithm-based fault tolerance for convolutional neural networks. *IEEE Transactions on Parallel and Distributed Systems* 32, 7 (2020), 1677–1689.

Appendix: Artifact Description

A Overview of Contributions and Artifacts

A.1 Paper's Main Contributions

C_1 End-to-End Fault Tolerance for Attention Mechanism:We implement module-level protection for Transformers by fusing the operations within the attention module into a single kernel with end-to-end fault tolerance, achieving higher efficiency than operation-level approaches.

C_2 Strided ABFT Without Inter-Thread Communication: For the linear computations in the attention and feed forward modules, we perform error detection using strided encoding and strided verification, eliminating the inter-thread communication overhead required by traditional ABFT schemes.

C_3 Computation-Aware Selective Protection: For the nonlinear softmax computations, we propose selective neuron value restriction, which applies fine-grained protection to computations critical for determining attention scores, while adopting relaxed protection for other operations.

C_4 Optimized Fault Tolerance with Unified Verification: Through unified verification, multiple computation steps are checked collectively, reducing the frequency of fault tolerance. Experimental results show that the optimized verification approach can achieve up to a 7.56× speedup over operation-level protection methods.

A.2 Computational Artifacts

A_1 https://doi.org/10.5281/zenodo.15881566

Artifact ID	Contributions Supported	Related Paper Elements
A_1	C_1	Section 4.1
	C_2	Section 4.2
	C_3	Section 4.3
	C_4	Section 4.4

B Artifact Identification

B.1 Computational Artifact A_1

Relation To Contributions

The artifact includes the source code of the FT-Transformer, with an emphasis on the end-to-end fault tolerant attention design. By running the provided project code, users can reproduce the performance and fault tolerance overhead results for the end-to-end fault tolerance framework, strided ABFT, selective neuron value restriction, and optimized unified protection. These results correspond to Figures 9–15 and Tables 1–2 in the paper.

Expected Results

C_1 The end-to-end fault tolerant attention is expected to be faster than existing operation-level protection methods for attention on GPU, and can support longer input sequences.

C_2 Strided ABFT demonstrates consistently lower protection overhead compared to traditional ABFT across all tested sequence lengths under typical Transformer configurations.

C_3 Under the same test condition as $C2$, selective neuron value restriction is faster than duplication-based methods.

C_4 The unified verification strategy incurs lower overhead than the unoptimized individual verification approach.

Expected Reproduction Time (in Minutes)

Reproducing this artifact is estimated to take approximately 45 minutes in total. This includes around 20 minutes for setting up the software environment, 10 minutes for compiling all test cases, and 15 minutes for executing the experiments individually.

Artifact Setup (incl. Inputs)

Hardware. This artifact is evaluated on a NVIDIA 40GB A100-PCIE GPU, which is connected to a node with a 64-core AMD EPYC7763 CPU.

Software. The software used in this artifact mainly includes the following packages.

Software	Version	URL
cuda toolkit	12.4	developer.nvidia.com/cuda-downloads
cmake	3.12	cmake.org/download/
gcc	5.0	gcc.gnu.org/install/download.html
python	3.9.19	www.python.org/downloads/
pytorch	2.3.0	pytorch.org/get-started/locally/
transformers	4.40.1	github.com/huggingface/transformers
gflags	2.2.2	github.com/gflags/gflags

Datasets / Inputs. During the execution of the test code, random high-dimensional tensors are automatically generated to serve as inputs to the attention module, representing the Query, Key, and Value matrices. These tensors have the shape of $batch \times head_num \times seq_len \times dim$, where the parameters can be configured through command-line arguments -b, -hq, -sq, and -d. The provided bash scripts have pre-configured the input size parameters for testing.

Installation and Deployment. Once all hardware and software setup has been completed, the FT-Transformer can be compiled using the following commands.

```
# clone the artifact for test
mkdir ft_transformer_test
cd ft_transformer_test
cp <path-to-the-artifact>/FT_Transformer.zip ./
unzip FT_Transformer.zip

# compile the FT_Transformer
cd FT_Transformer
bash build.sh -v Basic
# Usage: bash build.sh -v <Basic, ABFT, SNVR, Optimized>
```

The -v option in the build.sh script determines the version of the compiled executable, corresponding to the test codes required for $C1$ to $C4$.

Artifact Execution

We use the profile.sh script to evaluate the experiments for $C1$ to $C4$, following the same test procedure in all cases. Here, we use the C2 experiment as an example. The profile.sh script takes two parameters: -s for the testing scale and -v for the target version, which must match the version used during compilation.

```
# recompile the code for Strided ABFT evaluation
cd ft_transformer_test/FT_Transformer
bash build.sh -v ABFT

# profile the performance of Strided ABFT on Attention
bash profile.sh -s Huge -v ABFT
```

```
# Usage: -s <Huge, Small> -v <Basic, ABFT, SNVR, Optimized>
```

The -s option in profile.sh offers two predefined configurations: Huge and Small. The Huge parameter sets head_num=32 and dim=128, while Small sets head_num=16 and dim=64, corresponding to large and small transformer model settings. All profiling tests sweep input lengths from 512 to 16K, adjusting the batch size to maintain a constant total token count of 16K.

Artifact Analysis (incl. Outputs)

Experimental results will be saved in the `FT_Transformer/exp_final` directory after executing profile.sh.